The Fast Fourier Transform

1 Motivation: Multiplying Polynomials

Let

$$A(x) = a_0 + a_1 x + \cdots + a_d x^d, \qquad B(x) = b_0 + b_1 x + \cdots + b_d x^d.$$

Their product $C(x) = A(x)B(x)$ has degree $2d$, and computing all its coefficients directly (convolving the coefficient lists) takes $O(d^2)$ time. The FFT lets us do this in $O(d \log d)$ instead, by moving between two representations of a polynomial.

2 Two Representations of a Polynomial

2.1 Coefficient representation

$$A(x) = a_0 + a_1 x + \cdots + a_{n-1} x^{n-1}.$$

2.2 Value (point) representation

Given a list of n pairs $(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})$ with distinct x_i, there is *exactly one* polynomial A of degree $\leq n-1$ such that $A(x_i) = y_i$ for all i.

So we can equivalently describe A by its **value representation**: the set of n points

$$\big\{(x_0, A(x_0)),\ (x_1, A(x_1)),\ \ldots,\ (x_{n-1}, A(x_{n-1}))\big\}.$$

2.3 The two are related by a linear map

$$\begin{pmatrix} A(x_0) \\ A(x_1) \\ \vdots \\ A(x_{n-1}) \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ \vdots & & & & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{pmatrix}}_{M,\ n \times n} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix}.$$

This M is the **Vandermonde matrix**, $M_{ij} = x_i^j$.

> M is invertible $\iff$ all x_i are distinct. Invertibility guarantees we can always translate back and forth between the value representation and the coefficient representation without losing information.

3 Multiplying in the Value Representation

Suppose $\deg A = \deg B = d$, so $C = AB$ has $\deg C = 2d$. To pin down C uniquely we need $2d+1$ points.

Idea: evaluate A and B at the *same* $2d+1$ points $x_0, \ldots, x_{2d}$. Then

$$C(x_i) = A(x_i)\, B(x_i), \qquad i = 0, \ldots, 2d,$$

which is just $2d+1$ independent scalar multiplications — $O(d)$ time, once we already have the values $A(x_i)$ and $B(x_i)$.

So the overall plan is:

1. **Evaluate** A and B at $2d+1$ points (fast: FFT, $O(d \log d)$).
2. **Multiply** pointwise: $C(x_i) = A(x_i)B(x_i)$ $(O(d))$.
3. **Interpolate** back to recover the coefficients of C (fast: inverse FFT, $O(d \log d)$).

The only question is: which points make step 1 (and step 3) fast? Answer: the **roots of unity**.

4 Evaluating at the Roots of Unity

Let n be a power of 2. The n-th roots of unity are

$$\omega_n^k = e^{2\pi i k/n}, \qquad k = 0, 1, \ldots, n-1.$$

Split A into its even- and odd-indexed coefficients:

$$A(x) = A_e(x^2) + x\, A_o(x^2),$$

where A_e collects the even-degree coefficients of A and A_o the odd-degree coefficients (each a polynomial of degree $\leq n/2 - 1$).

4.1 Key relation, part 1: $k = 0, \ldots, n/2 - 1$

Using $\omega_n^2 = \omega_{n/2}$ (so $\omega_n^{2k} = \omega_{n/2}^k$):

$$A(\omega_n^k) = A_e\big(\omega_{n/2}^k\big) + \omega_n^k\, A_o\big(\omega_{n/2}^k\big). \tag{1}$$

4.2 Key relation, part 2: shift by $n/2$

Now consider $A(\omega_n^{k+n/2})$. Since $\omega_n^n = 1$:

$$\omega_n^{2(k+n/2)} = \omega_n^{2k+n} = \omega_n^{2k} \cdot \omega_n^n = \omega_n^{2k} = \omega_{n/2}^k, \qquad \omega_n^{k+n/2} = -\omega_n^k.$$

So

$$A\big(\omega_n^{k+n/2}\big) = A_e\big(\omega_{n/2}^k\big) - \omega_n^k\, A_o\big(\omega_{n/2}^k\big). \tag{2}$$

Two key relations, for $k = 0, \ldots, n/2 - 1$:

$$A(\omega_n^k) = A_e\big(\omega_{n/2}^k\big) + \omega_n^k A_o\big(\omega_{n/2}^k\big), \qquad A\big(\omega_n^{k+n/2}\big) = A_e\big(\omega_{n/2}^k\big) - \omega_n^k A_o\big(\omega_{n/2}^k\big).$$

So: if we know A_e and A_o evaluated at the $\frac{n}{2}$-th roots of unity $\{\omega_{n/2}^k : k = 0, \ldots, n/2-1\}$, we get A evaluated at *all* n points $\{\omega_n^k : k = 0, \ldots, n-1\}$ using only $O(n)$ extra work.

4.3 Recursion

A_e and A_o are each polynomials of degree $\leq n/2 - 1$. Evaluating a degree-$(n/2-1)$ polynomial at the $(n/2)$-th roots of unity is *exactly the same problem*, one level smaller. So we can recurse: apply the same trick to A_e and A_o, then to their own even/odd halves, and so on.

$$T(n) = 2T(n/2) + O(n) \;=\; O(n \log n).$$

5 Pseudocode: Forward FFT

FFT(A, n) # A is a coefficient array of length n, with $n = 2^m$

if $n = 1$: **return** A

$A_e = [a_0, a_2, a_4, \dots]$

$A_o = [a_1, a_3, a_5, \dots]$

$Y_e =$ **FFT**$(A_e,\ n/2)$

$Y_o =$ **FFT**$(A_o,\ n/2)$

for $k = 0$ **to** $n/2 - 1$:

$\omega = e^{2\pi i k/n}$

$Y[k] \ = \ Y_e[k] + \omega\, Y_o[k]$

$Y[k + n/2] \ = \ Y_e[k] - \omega\, Y_o[k]$

return Y

6 Interpolation: Going Back from Values to Coefficients

From Section 2.3, with $x_k = \omega_n^k$ we can write $M_{jk} = \omega_n^{jk}$, so

$$\begin{pmatrix} A(\omega_n^0) \\ \vdots \\ A(\omega_n^{n-1}) \end{pmatrix} = M \begin{pmatrix} a_0 \\ \vdots \\ a_{n-1} \end{pmatrix}, \qquad M_{jk} = \omega_n^{jk}.$$

To go from values $y = (A(\omega_n^0), \dots, A(\omega_n^{n-1}))$ back to coefficients, we need M^{-1}. A standard fact about the DFT matrix is

$$M^{-1} = \frac{1}{n}\overline{M} = \frac{1}{n}\left[\omega_n^{-jk}\right],$$

so that

$$a_j = \frac{1}{n}\sum_{k=0}^{n-1} y_k\, \omega_n^{-jk}.$$

This is *the same FFT algorithm*, just run with $\omega = e^{-2\pi i/n}$ instead of $e^{2\pi i/n}$, and the output divided by n:

IFFT(y, n):

$a =$ **FFT**$(y,\ n,\ \omega = e^{-2\pi i/n})$

return $(a_j/n$ for each a_j in $a)$

7 Putting It Together: Fast Polynomial Multiplication

Given A, B with $C = AB$ of degree at most $2n - 2$ (each factor of degree $\le n - 1$):

1. Choose N, the smallest power of 2 with $N \geq 2n - 1$.
2. **Pad** A and B with zeros up to length N (treat them as degree $N - 1$ polynomials).
3. **Evaluate** both at the N-th roots of unity: $\omega_N^k = e^{2\pi ik/N}$, $k = 0, \ldots, N - 1$ (via FFT).
4. **Multiply pointwise:**

$$C(\omega_N^k) = A(\omega_N^k)\, B(\omega_N^k), \qquad k = 0, \ldots, N - 1.$$

   This gives C's value representation at N points. Since $N \geq 2n - 1$, this is enough to uniquely determine the degree-$(2n - 2)$ polynomial C.
5. **Interpolate** (via IFFT) to recover C's coefficients.

Total runtime: $O(N \log N) = O(n \log n)$ — a substantial improvement over the naive $O(n^2)$ convolution.